Firjan SENAI

Situação de Aprendizagem

SITUAÇÃO DE APRENDIZAGEM	
Segmento:	Técnico
Unidade (s) Curricular (es):	Desenvolvimento de Sistemas
Curso (s):	Desenvolvimento de Sistemas
Instrutor (a):	Vitor Hugo Almeida de Sousa

ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA	

[x] Situação-problema [ ] Estudo do Caso [ ] Projeto [ ] Pesquisa Aplicada

DESCRIÇÃO DA ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA

Contextualização: Cadastro de Setores e Equipamentos para Ordens de Serviço

Uma equipe de suporte precisa conhecer os setores e os equipamentos atendidos antes de registrar ordens de serviço. Você iniciará uma API Spring Boot para cadastrar e consultar esses dados. A abertura de ordens de serviço será desenvolvida em outra etapa.

Nesta atividade, você seguirá o caminho Model → Repository → Service → Controller apresentado no material DEV_SIS_33_36. Use MySQL e Postman para confirmar cada operação.

DESAFIO

Etapa 1 — Criar e executar o projeto

No Spring Initializr, siga o procedimento do material DEV_SIS_9: Maven, Java, Jar, versão estável do Spring Boot e dependências Spring Web, Spring Data JPA, Validation e MySQL Driver. Use o artifact **aberturaordensservico** e o pacote principal **br.com.ordensservico**.

Abra o projeto na IDE. Crie o banco no MySQL com **CREATE DATABASE ordens_servico**; Configure a conexão no application.properties, incluindo spring.jpa.hibernate.ddl-auto=update. O JPA criará as tabelas, mas o banco deve existir.

Etapa 2 — Registrar o projeto no GitHub

Crie no GitHub um repositório vazio chamado **abertura-ordens-servico**. Abra o git bash na pasta do projeto. Confira o .gitignore e remova a linha que contém .mvn/wrapper/maven-wrapper.jar.

No git bash execute, nesta ordem:

1. git init
2. git add .
3. git commit -m "chore: iniciar projeto Spring Boot"
4. git branch -M main
5. git remote add origin https://github.com/SEU_USUARIO/abertura-ordens-servico.git
6. git push -u origin main.

Substitua SEU_USUARIO; se origin já existir, confira com git remote -v antes de adicioná-lo.

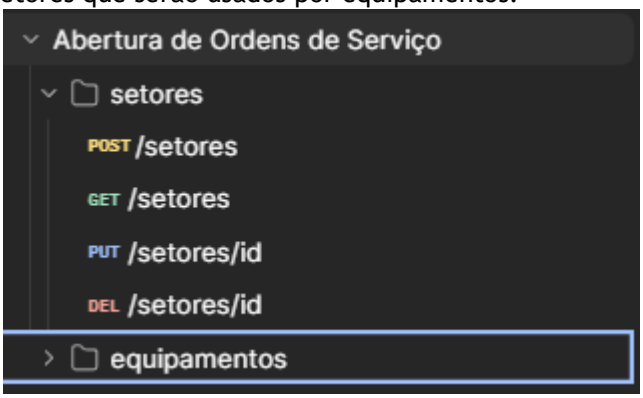
Etapa 3 — Planejar os dados e o Postman

Crie a collection “Abertura de Ordens de Serviço” no Postman e organize as requisições em pastas, /setores e /equipamentos.

Etapa 4 — Implementar Setor na ordem das camadas

1. **Model**: no pacote model, crie **Setor.java** com @Entity, @Table(name = "setor"), @Id e @GeneratedValue para id, além de nome. Confira a criação da tabela ao iniciar a aplicação.
2. **Repository**: no pacote repository, crie a interface SetorRepository estendendo JpaRepository<Setor, Integer>.
3. **Service**: em SetorService, receba o Repository pelo construtor e organize cadastrar (save), listar (findAll), buscarPorId (findById), atualizar (consultar e salvar) e excluir (verificar existência e remover).
4. **Controller**: em SetorController, receba o Service pelo construtor, use /setores como rota principal e disponibilize POST /setores, GET /setores, GET /setores/{id}, PUT /setores/{id} e DELETE /setores/{id}. Siga os retornos 201, 200, 204 e 404 do material; para PUT, envie todos os campos editáveis.

5. No Postman, salve e teste POST http://localhost:8080/setores com Body → raw → JSON: {"nome":"Costura"}. Cadastre também {"nome":"Montagem"}. Faça GET da lista, GET por um ID existente e por 999. Faça PUT em um ID existente com {"nome":"Costura de Acabamento"} e depois GET para conferir. Teste DELETE em um setor criado apenas para exclusão e confirme 404 Not Found ao consultar novamente o ID excluído. Não exclua os setores que serão usados por equipamentos.



6. Após todos os testes, adicione essa funcionalidade ao histórico do git
   git add .
   git commit -m “feat crud de setores”

Etapa 5 — Implementar Equipamento na mesma ordem

1. **Model**: crie Equipamento.java com @Entity, @Table(name = "equipamento"), id, nome, numeroPatrimonio e setor. Use @ManyToOne no atributo setor.
2. **Repository:** crie EquipamentoRepository estendendo JpaRepository<Equipamento, Integer>. Confira no MySQL a tabela e sua coluna de associação ao setor.
3. **Service**: crie EquipamentoService com cadastrar, listar, buscarPorId, atualizar e excluir, usando o Repository. Antes de salvar um equipamento, confirme que o ID do setor informado existe por meio de SetorRepository; associe o setor encontrado. Se não existir, devolva uma resposta apropriada de erro, sem gravar o equipamento. Na atualização, resolva novamente o setor informado.
4. **Controller**: crie EquipamentoController com rota principal /equipamentos e POST, GET da lista, GET por ID, PUT por ID e DELETE por ID. Use o padrão do Controller de Setor. Mantenha a relação em apenas um sentido para que as respostas JSON sejam simples.

   No Postman, salve POST /equipamentos com Body → raw → JSON:

```
{
   "nome": "Máquina de Costura Industrial",
   "numeroPatrimonio": "PAT-001",
   "setor": {
        "id":1
    }
}
```

Troque 1 pelo ID real do setor Costura. Cadastre outro equipamento, como uma Prensa de Montagem, com patrimônio PAT-002, associado ao setor Montagem.

Salve GET /equipamentos e GET /equipamentos/{id}. Confira no retorno de cada equipamento o setor correspondente e, no MySQL, o vínculo gravado. Faça PUT /equipamentos/{id} com todos os campos editáveis, por exemplo: {"nome":"Máquina de Costura Industrial Revisada","numeroPatrimonio":"PAT-001","setor":{"id":2}}, substituindo 2 pelo ID real de outro setor cadastrado, como Costura de Acabamento. Repita o GET para confirmar a atualização e a troca de setor.

Teste GET por ID inexistente e POST /equipamentos com {"nome":"Máquina de Corte","numeroPatrimonio":"PAT-003","setor":{"id":999}}. Confirme que o setor 999 não existe e verifique que o equipamento não foi cadastrado. Crie um equipamento apenas para testar DELETE /equipamentos/{id}; depois, consulte o mesmo ID com GET e confirme o retorno 404 Not Found. Salve todas as requisições e suas descrições na collection.

RESULTADOS E ENTREGAS ESPERADOS
Entregue o link do repositório abertura-ordens-servico no GitHub, com o código atualizado, commits das etapas e instruções para executar o projeto e criar o banco ordens_servico, sem divulgar credenciais.
Exporte e entregue a collection “Abertura de Ordens de Serviço”, com as pastas Teste, Setores e Equipamentos, rotas organizadas, documentadas e testadas. A demonstração deve mostrar a associação de cada equipamento a um setor existente e a resposta ao informar um setor inexistente.

Confira antes da entrega: GET /teste responde; os dois CRUDs funcionam; a associação persiste no MySQL; há testes de consulta e exclusão por ID inexistente; o último commit foi enviado com git push.	
Material de Consumo:	Não se aplica
Ferramentas:	IDE; MySQL Workbench; Postman; Git e GitHub; Spring Boot
EPIs:	Não se aplica
EPCs:	Não se aplica
Equipamentos:	Computador com Java/JDK, MySQL e acesso às dependências do projeto
Instrumentos:	Não se aplica